따라하며 배우는

빅데이터 분석

Big Data Analytics

박동규 지음

CHAPTER

04 넘파이의 세계로

CONTENTS

4.1 다차원 배열의 최대값, 최소값, 평균값 구하기와 정렬

4.2 평균값과 중앙값을 알아보자

LAB 4-1 입력값의 합, 최대값, 최소값, 평균값 출력하기

LAB 4-2 다차원 배열의 평균 구하기와 정렬하기

4.3 중앙값과 최빈값을 알아보자

4.4 분산과 편차를 알아보자

4.5 평균, 분산, 표준편차를 알아보자

LAB 4-3 입력값의 평균, 분산, 표준편차 구하기

LAB 4-4 어느 모둠의 분산이 가장 큰가

이 장에서 배울 것들

- 넘파이에서 다차원 배열을 위해 지원하는 여러 가지 연산을 알아봅시다.
- 평균값, 중앙값, 최빈값을 구하는 방법을 익혀봅시다.
- 넘파이를 통해 평균, 표준편차, 분산을 구하는 방법을 익혀봅시다.

4.1 다차원 배열의 최대값, 최소값, 평균값 구하기와 정렬

- [10, 20, 30]의 세 원소를 가진 1차원 배열의 max(), min(), mean() 메소드는 최대값 30, 최소값 10, 평균값 20.0을 각각 출력
- 넘파이 함수로는 np.max(), np.min(), np.mean()으로 구현되어 있음
- 원소들의 자료형을 변환하고자 한다면 astype() 메소드를 사용

```
a = np.array([10, 20, 30])
a.max(), a.min()        # a 배열원소 중 최대값, 최소값을 출력함
```

```
(30, 10)
```

```
a.mean()                # a의 평균을 출력함
```

```
20.0
```

```
np.max(a), np.min(a), np.mean(a)    # 넘파이 함수를 사용하여 동일한 기능을 구현
```

```
(30, 10, 20.0)
```

```
a.astype(np.float64)  # 원소의 자료형을 float64로 변환
```

```
array([10., 20., 30.])
```

4.1 다차원 배열의 최대값, 최소값, 평균값 구하기와 정렬

- 넘파이 다차원 배열 역시 파이썬의 리스트와 마찬가지로 정렬을 위한 sort() 함수를 가지고 있음
- 다음과 같은 6개의 원소를 가진 다차원 배열을 오름차순으로 정렬하고 역순 인덱싱을 통해 내림차순으로 각각 정렬된 결과를 출력하자.

```
c = np.array([35, 24, 55, 69, 19, 99])
c.sort()        # 배열의 정렬(오름차순)
c
```

```
array([19, 24, 35, 55, 69, 99])
```

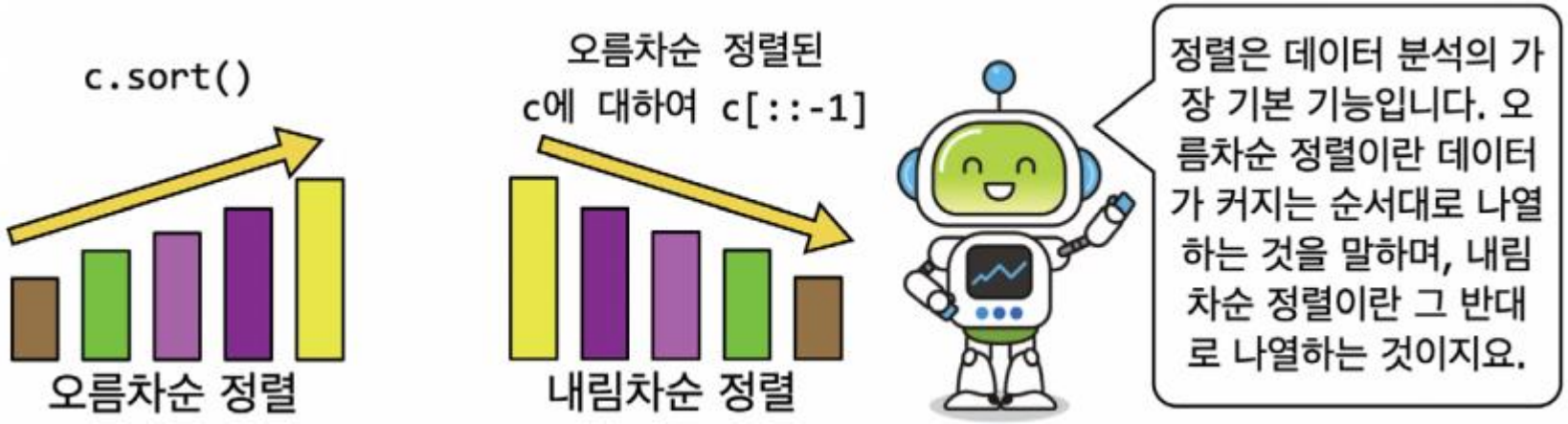
```
d = c[::-1]      # 오름차순 배열을 내림차순으로 정렬
d
```

```
array([99, 69, 55, 35, 24, 19])
```

```
c_flip = np.flip(c)  # flip() 함수를 사용하여 내림차순으로 정렬
c_flip
```

```
array([99, 69, 55, 35, 24, 19])
```

4.1 다차원 배열의 최대값, 최소값, 평균값 구하기와 정렬



4.1 다차원 배열의 최대값, 최소값, 평균값 구하기와 정렬

- 2차원 배열에 sort() 메소드를 사용하면 원소들은 기본적으로 axis 1 방향으로 오름차순 정렬됨
- 키워드 인자로 axis=0을 넣어주어야만 axis 0의 방향으로 오름차순 정렬됨

```
e = np.array([[35, 24, 55], [69, 19, 9], [4, 1, 11]])
e.sort()              # 배열의 정렬, 디폴트 axis는 1
e
```

```
array([[24, 35, 55],
       [ 9, 19, 69],
       [ 1,  4, 11]])
```

```
f = np.array([[35, 24, 55], [69, 19, 9], [4, 1, 11]])
f.sort(axis=0)        # axis=0 방향 정렬
f
```

```
array([[ 4,  1,  9],
       [35, 19, 11],
       [69, 24, 55]])
```

4.1 다차원 배열의 최대값, 최소값, 평균값 구하기와 정렬

- 배열 원소들을 정렬한 다음 넘파이의 flip() 함수를 사용하여 특정한 축에 대하여 값을 뒤집어 나열해 보자.

```
e = np.array([[35, 24, 55], [69, 19, 9], [4, 1, 11]])
e.sort()          # 배열의 정렬, 디폴트 axis는 1
print('--- flip() 이전 ---')
print(e)
g = np.flip(e, axis=0)
print('--- flip(e, axis=0) 이후 ---')
print(g)
h = np.flip(e, axis=1)
print('--- flip(e, axis=1) 이후 ---')
print(h)
```

```
--- flip() 이전 ---
[[24 35 55]
 [ 9 19 69]
 [ 1  4 11]]
--- flip(e, axis=0) 이후 ---
[[ 1  4 11]
 [ 9 19 69]
 [24 35 55]]
--- flip(e, axis=1) 이후 ---
[[55 35 24]
 [69 19  9]
 [11  4  1]]
```

4.1 다차원 배열의 최대값, 최소값, 평균값 구하기와 정렬

도전문제 4.1 다차원 배열의 내림차순 정렬

난이도 중

위의 결과를 수정하여 정렬되기 이전의 e 배열의 값에 대하여 axis 0 방향에 대하여 각각 내림차순 정렬이 되도록 하고 그 결과를 다음과 같이 출력하여라.

```
array([[69, 24, 55],
       [35, 19, 11],
       [ 4,  1,  9]])
```

4.2 평균값과 중앙값을 알아보자

- 8명의 사람을 대상으로 키(cm)를 측정하였다고 가정할 때, 데이터가 만일 다음과 같이 이루어져 있다면 이 데이터에 대해서 우리는 어떠한 정보를 얻을 수 있을까?

```
heights = np.array([ 163, 186, 165, 176, 169, 181, 183, 172])
heights.size
```

```
8
```

- 측정한 데이터들은 수치값의 연속에 불과하므로 우리는 데이터의 의미를 한눈에 이해하기 힘듦
- 하지만 이들의 합을 구하여 8로 나눈다면 우리는 키 데이터가 대략 174cm 근처에 많이 위치하고 있음을 알 수 있음
- 넘파이의 sum() 함수는 다차원 배열의 합을 반환하며, mean() 함수는 다차원 배열의 평균을 반환

4.2 평균값과 중앙값을 알아보자

```
print('키 값들의 합 :', np.sum(heights))
print('키 값들의 평균 :', np.sum(heights)/heights.size)
```

```
키 값들의 합 : 1395
키 값들의 평균 : 174.375
```

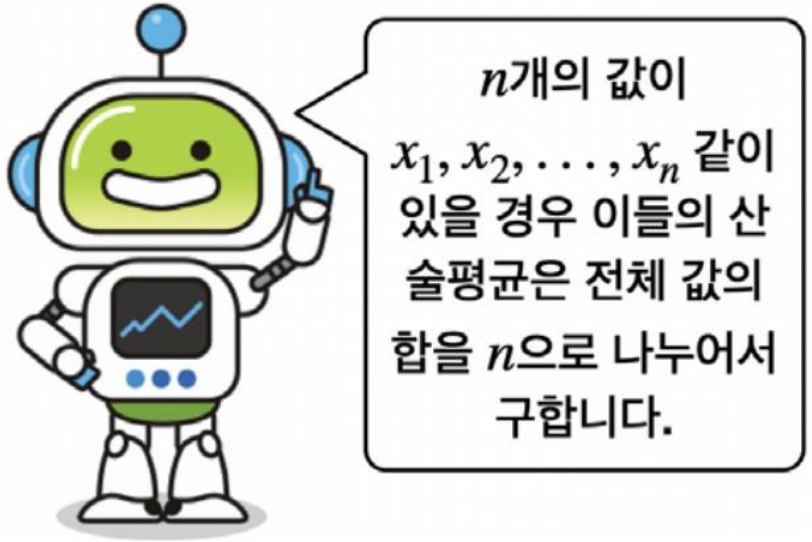
```
print('키 값들의 평균 :', np.mean(heights)) # 위와 동일한 결과
```

```
키 값들의 합 : 1395
키 값들의 평균 : 174.375
```

4.2 평균값과 중앙값을 알아보자

산술평균을 구하는 공식

$$= \frac{x_1 + x_2 + \ldots + x_n}{n}$$



- **평균**mean이란 **데이터의 중심에 있는 수치로 데이터의 전체적인 경향을 나타내는 정보**
- 데이터들의 합을 데이터의 개수로 나눈 값을 통계학에서는 **산술평균**arithmetic mean이라는 용어를 사용하기도 함
- 평균과 같이 데이터의 전체적인 경향을 나타내는 정보를 **대표값**이라고 함

4.2 평균값과 중앙값을 알아보자

- 산술평균은 좋은 대표값이기는 하지만 이상치(outlier)와 같은 극단적인 값들에 대하여 민감하게 움직인다는 점에서 문제점이 존재

상황: 어느 직장에 근무하는 10명의 사람들의 연봉을 조사했고, 그 결과는 다음과 같다(단위: 만 달러).
연봉: 30, 35, 40, 45, 50, 55, 60, 65, 70,1000
분석

1. 많은 사람들의 연봉은 30만에서 70만 달러 사이이다. 하지만 한 사람의 연봉이 1000만 달러이다.
2. 이에 대한 산술평균을 계산하면 (30+35+40+45+50+55+60+65+70+1000)/10=145
3. 산술 평균값은 145만 달러이다.
4. 이 산술평균 145만 달러는 대부분 사람들의 연봉을 대표하지 못한다.
5. 10명 중에서 9명의 사람들 즉 대부분의 사람들이 이보다 훨씬 낮은 연봉을 받고 있다.

- 위의 경우와 같이 **산술평균값은 매우 큰 값에 대해서 민감하게 움직여서 전체를 대표하는 대표값으로 적절하지 못한 경우도 있음**

4.2 평균값과 중앙값을 알아보자

평균과 중앙값 계산하기

- 위의 상황을 넘파이를 이용하여 코딩해 보고 평균값과 **중앙값**median을 계산해보자.
- 중앙값이란 **다차원 배열의 값을 가장 작은 값부터 큰 값까지 늘어 놓았을 때 가장 가운데에 있는 항목**을 말하며, median() 함수를 사용하여 구한다.

```
earnings = np.array([30, 35, 40, 45, 50, 55, 60, 65, 70,1000])
print('연봉의 평균값 :', np.mean(earnings),'만 달러')
print('연봉의 중앙값 :', np.median(earnings),'만 달러')
```

```
연봉의 평균값 : 145.0 만 달러
연봉의 중앙값 : 52.5 만 달러
```

```
print('연봉 데이터를 크기순으로 정렬')
np.sort(earnings)
```

```
연봉 데이터를 크기순으로 정렬
array([   30,    35,    40,    45,    50,    55,    60,    65,    70,  1000])
```

4.2 평균값과 중앙값을 알아보자

평균과 중앙값 계산하기

- 중앙값은 이와 같이 10개의 수를 크기 순서대로 나열하였을 때, 가장 가운데 있는 항목인 50과 55의 평균인 52.5가 됨
- 중앙값은 전체 데이터에서 1000과 같이 비정상적으로 큰 값으로 인하여 전체의 평균값이 대표성을 가지지 못하는 경우를 막기 위해서 사용됨
- 이와 같이 **비정상적으로 큰 값이나 작은 값을 지칭하여 이상치**outlier라고 함

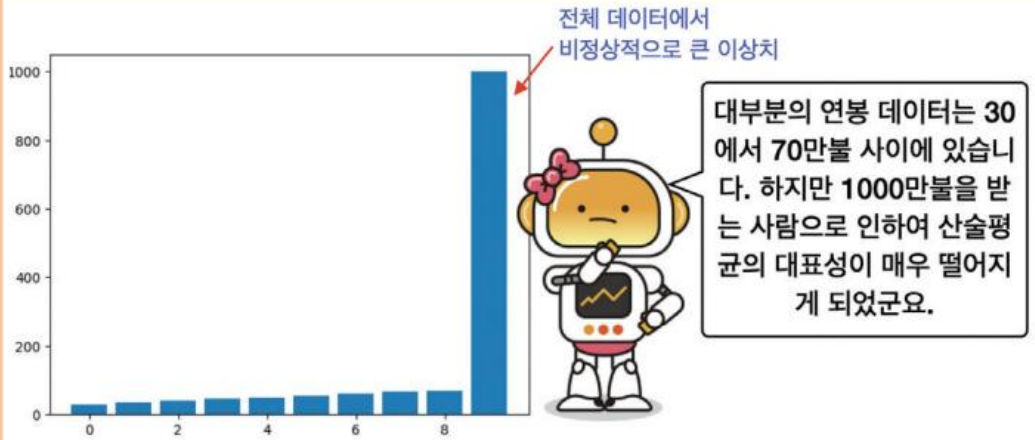
4.2 평균값과 중앙값을 알아보자

이상치를 알아보자

- 후에 다루게 될 matplotlib은 이러한 이상치를 시각적으로 나타내는데 매우 유용함
- matplotlib을 이용하여 데이터를 막대 차트로 나타내는 코드

```
import matplotlib.pyplot as plt

sorted_earnings = np.sort(earnings) # 데이터를 정렬
x = np.arange(10)                   # x 구간의 10개 값 지정
plt.bar(x, sorted_earnings)         # 막대 차트 그리기
```



LAB 4-1 입력값의 합, 최대값, 최소값, 평균값 출력하기

실습 시간

사용자로부터 임의의 정수를 다음과 같이 연속적으로 입력받도록 하자. 이들 정수는 공백 단위로 구분된다. 키보드 엔터키가 입력되면 입력이 종료되며, 이 입력값에 대하여 전체의 합, 최대값, 최소값, 평균값을 다음과 같이 출력하도록 하자.
이 기능은 넘파이의 함수를 사용하도록 하자.

원하는 결과

```
정수를 입력하시오 : 45 67 30 40 21 99 339 210
입력값들의 합 : 851
최대값 : 339
최소값 : 21
평균값 : 106.375
```

힌트

사용자로부터 공백 단위로 입력 받은 정수를 리스트에 저장하는 기능은 다음 코드를 사용하여 구현할 수 있다. 이 기능은 input() 함수로 입력받은 문자열을 split() 메소드로 구분한 다음 map 함수를 이용하여 정수형(int)으로 변환하는 기능이다.

```
arr = list(map(int,input('정수를 입력하시오 : ').split()))
arr
```

```
정수를 입력하시오 : 45 67 30 40 21 99 339 210
[45, 67, 30, 40, 21, 99, 339, 210]
```

이와 같은 명령으로 입력받은 정수값에 대하여 넘파이의 sum(), max(), min(), mean() 함수를 사용하면 이 기능을 구현할 수 있을 것이다.

LAB4-2 다차원 배열의 평균 구하기와 정렬하기

실습 시간

어떤 병원에서 4명의 환자들 대상으로 키와 몸무게를 측정하여 다음과 같이 2차원 넘파이 배열에 저장하였다.
이 다차원 배열에서 환자들의 키의 평균, 몸무게의 평균값을 각각 구하여라.

```
patients = np.array([[ 1.63, 1.86, 1.63, 1.76],  # 환자들의 키
                     [ 76.0, 84.0, 69.0, 85.0]]) # 환자들의 몸무게
```

이 값을 다음과 같이 출력하여라.

원하는 결과

```
환자들의 키의 평균 : 1.72
키를 크기 순으로 정렬 : [1.63 1.63 1.76 1.86]
환자들의 몸무게의 평균 : 78.5
몸무게를 크기 순으로 정렬 : [69. 76. 84. 85.]
```

힌트

patients는 2행 4열의 다차원 배열이다. 이 다차원 배열에서 키 행만을 출력하려면 patients[0]를 이용하면 된다. 마찬가지로 몸무게 행을 출력하려면 patients[1]을 사용한다.

넘파이 다차원 배열의 값에 대하여 평균을 구하는 함수는 mean()이다. 또한 다차원 배열의 값을 오름차순으로 정렬하는 명령은 sort()이다. 이 명령들을 사용하여 주어진 기능을 구현하도록 하자.

4.3 중앙값과 최빈값을 알아보자

- **중앙값은 이상치의 영향을 덜 받기 때문에, 데이터의 실제 중심 경향을 더 잘 나타내는 경우가 많음**
- 평균, 중앙값과 함께 사용될 수 있는 또 다른 대표값으로는 **최빈값**mode이 있음
- 최빈값은 데이터 집합에서 가장 자주 나타나는 값
- 예를 들어, 데이터 집합이 [2, 3, 4, 4, 5, 5, 5, 6, 6]라면, 5가 세 번으로 가장 많이 나타나므로 이 데이터 집합의 최빈값은 5라고 할 수 있음

[2, 3, 4, 4, 5, 5, 5, 6, 6]

4.3 중앙값과 최빈값을 알아보자

- 최빈값은 수치 데이터뿐만 아니라, 범주형 데이터(예: 색깔, 브랜드, 이름 등)에도 적용될 수 있음

- 예를 들어, **'빨강'**, **'파랑'**, **'녹색'**, **'파랑'** 색상이 있을 때, **'파랑'**이 두 번 나타나 가장 빈번하므로 최빈값은 **'파랑'**

- 이처럼 **최빈값은 데이터 집합 내에서 가장 일반적이거나 흔한 경우를 파악할 때 유용함**

4.3 중앙값과 최빈값을 알아보자

- 넘파이에서 최빈값을 구하기 위해서는 bincount() 라는 함수를 먼저 사용해야 함
- bincount() 함수는 **각 자료값의 빈도수를 반환하는 함수**
- 이 때 빈도수는 0부터 이 집합의 최대값 6까지의 빈도수를 차례로 출력
- 예의 경우에서 0이 0번, 1이 0번, 2가 1번, 3이 1번,..., 6이 2번 나타나므로 그 결과는 다음과 같음

```
a = np.array([2, 3, 4, 4, 5, 5, 5, 6, 6])
# a에 있는 값들 중 0에서 6사이 값의 빈도수를 차례로 출력
np.bincount(a)
```

```
array([0, 0, 1, 1, 2, 3, 2])
```

4.3 중앙값과 최빈값을 알아보자

- 넘파이의 argmax() 함수는 **넘파이 다차원 배열에서 가장 큰 값의 인덱스를 반환**함

- 따라서 argmax([0, 0, 1, 1, 2, 3, 2])는 가장 큰 값 3의 인덱스 5를 반환함

```
np.argmax([0, 0, 1, 1, 2, 3, 2]) # a 다차원 배열에서 가장 큰 값의 인덱스를 반환
```

```
5
```

```
mode = np.argmax(np.bincount(a)) # 위와 동일한 값 5를 mode에 저장
```

- 이제 위의 코드를 활용하여 다음과 같이 최빈값을 출력해 보고, 그 빈도도 나타내어 보자.

```
print('배열', a,'의 최빈값은 ', mode, '이며', np.bincount(a)[mode],'번 나타남')
```

```
배열 [2 3 4 4 5 5 5 6 6] 의 최빈값은  5 이며 3 번 나타남
```

4.3 중앙값과 최빈값을 알아보자

- 최빈값을 구하기 위한 더 효율적인 방법은 사이파이(scipy)라는 파이썬 패키지에 있는 stats 서브모듈의 mode() 함수를 사용하는 것
- **사이파이는 과학 기술 계산을 위한 파이썬 라이브러리**
- mode() 함수는 **주어진 다차원 배열의 최빈값과 최빈값의 출현 횟수도 반환**

```python
# scipy 모듈의 stats 서브모듈을 가져온다
from scipy import stats
import numpy as np

# stats 모듈의 mode() 함수의 결과를 출력하자
print(stats.mode(a))
```

```
ModeResult(mode=5, count=3)
```

```python
mode = stats.mode(a)
print('배열', a,'의 최빈값은 ', mode.mode, '이며', mode.count,'번 나타남')
```

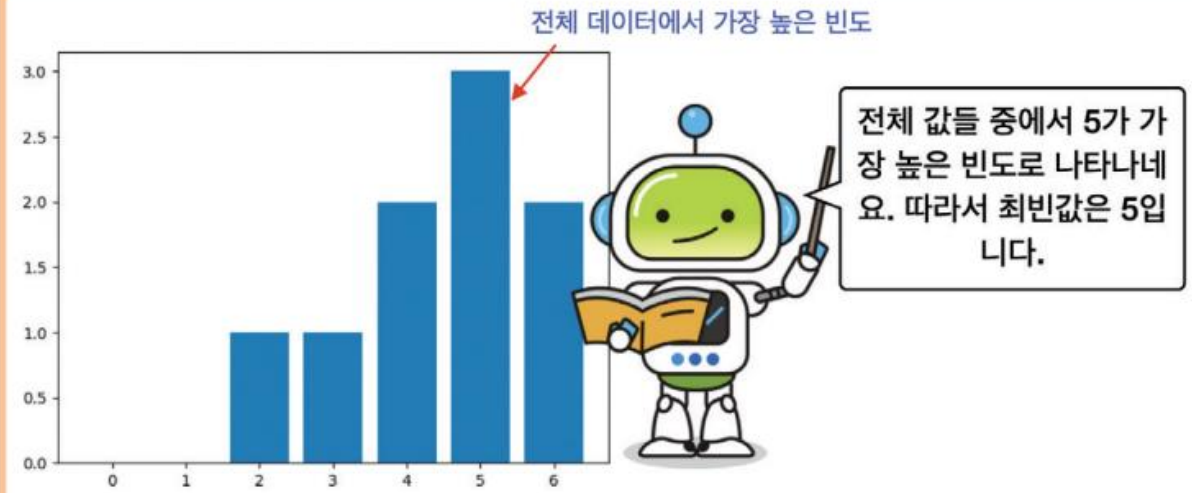
```
배열 [2 3 4 4 5 5 5 6 6] 의 최빈값은  5 이며 3 번 나타남
```

4.3 중앙값과 최빈값을 알아보자

- matplotlib 코드를 이용하여 결과를 시각화

```
import matplotlib.pyplot as plt

a = np.array([2, 3, 4, 4, 5, 5, 5, 6, 6])
# a에 있는 값들 중 0에서 6사이 값의 빈도수를 차례로 출력
bins_a = np.bincount(a)
x = np.arange(7)          # x 구간의 0에서 6까지 값 지정
plt.bar(x, bins_a)        # 막대 차트 그리기
```



4.3 중앙값과 최빈값을 알아보자

NOTE 라이브러리, 모듈, 패키지를 알아보자

파이썬은 함수나 상수, 변수, 클래스를 파일로 만들어 저장한 후 다른 파일에서 불러서 사용할 수 있다. 이러한 파이썬 파일은 .py라는 확장자를 가지는데 이 파일을 **모듈**module이라고 부른다. 이 모듈은 특정한 기능을 수행하기 위하여 필요에 따라 여러가지로 만들어 두기도 한다. 이와 같이 특정한 기능과 관련된 여러 모듈을 한 폴더 안에 두면 이를 쉽게 관리할 수 있는데 이것을 **패키지**package라고 한다. 하나의 패키지 안에는 하나 이상의 파이썬 파일들이나 리소스가 포함될 수 있다.

파이썬 **라이브러리**library는 패키지보다 더 큰 개념으로 여러가지의 패키지와 모듈을 모아둔 것을 말한다. 앞서 다룬 사이파이(Scipy), 맷플롯립(matplotlib)은 여러 모듈을 포함하고 있으므로 패키지로 불러야 하지만 편의상 라이브러리라고 지칭하는 경우도 많다.

4.4 분산과 편차를 알아보자

- 데이터를 분석할 때 가장 널리 사용되는 평균값은 **데이터를 대표하는 값**을 알아보는데 도움이 됨

- 평균값은 매우 유용한 정보이지만, 다음과 같은 상황에서는 평균값만으로는 데이터를 완전히 분석하였다고 보기 힘듦

상황: A와 B 모둠의 학생들이 수학 시험을 봤고, 그 결과가 다양하게 나왔다고 가정해보자. 이때, 시험 점수의 평균값만으로는 모둠 전체의 성적 분포를 완전히 이해하기 어렵다.

A 모둠의 성적값: 60, 70, 80, 90, 100
B 모둠의 성적값: 76, 78, 80, 82, 84

두 모둠 모두 성적의 평균이 80이다. 하지만 A 모둠의 학생들은 성적이 매우 낮은 학생들과 성적이 매우 뛰어난 학생들이 섞여 있는 모둠이다. 반면 B 모둠의 학생들은 성적이 80점 근처에 있는 학생들이 대부분이며 매우 낮은 성적의 학생과 매우 높은 성적의 학생은 없다.

4.4 분산과 편차를 알아보자

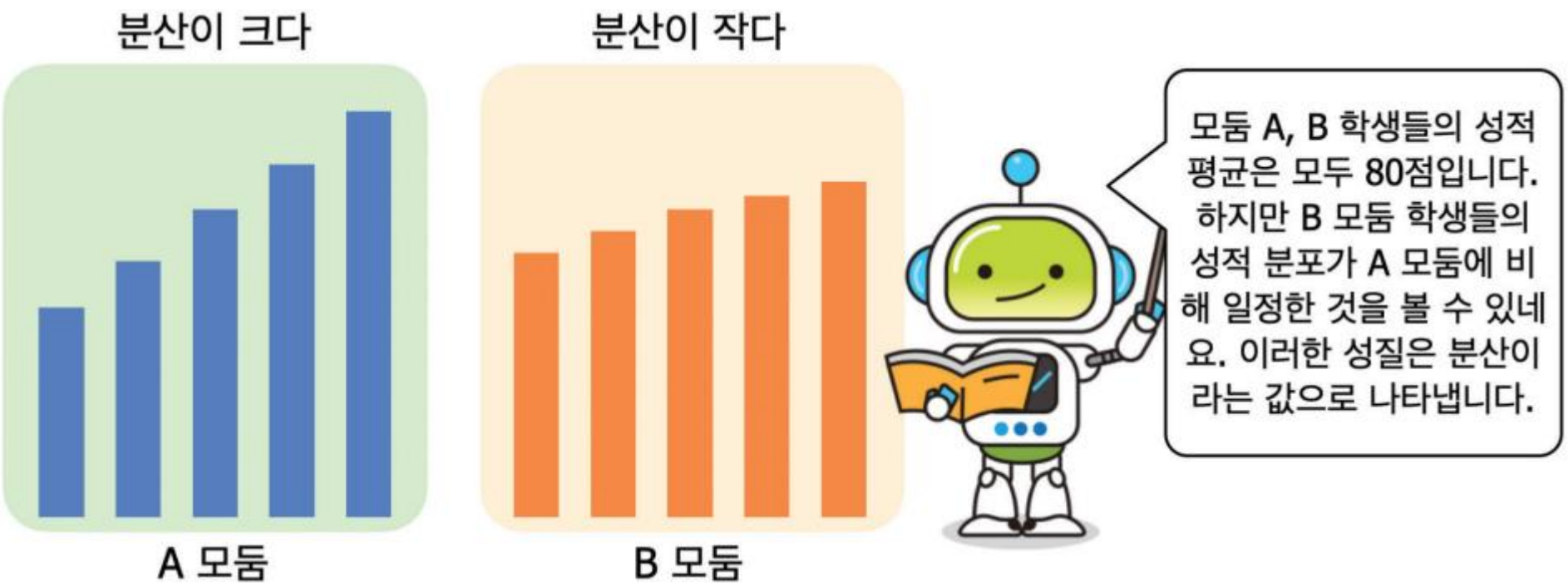
- 위의 상황을 다음과 같은 코드로 살펴보도록 하자.

```
group_A = np.array([60, 70, 80, 90, 100])
group_B = np.array([76, 78, 80, 82, 84])
print('A 모둠 학생들의 수학 점수 평균 :', np.mean(group_A))
print('B 모둠 학생들의 수학 점수 평균 :', np.mean(group_B))
```

```
A 모둠 학생들의 수학 점수 평균 : 80.0
B 모둠 학생들의 수학 점수 평균 : 80.0
```

- 두 모둠 학생들의 수학 점수 평균은 80점으로 같으나, 두 그룹의 대표값이 같다고 해서 두 그룹의 성격이 동일하다고 볼 수는 없음
- 이 경우 두 그룹의 **분산**variance을 계산하면, 학생들의 점수가 평균값에서 얼마나 멀리 퍼져 있는지, 즉 **점수의 일관성과 균일성을 파악**할 수 있음

4.4 분산과 편차를 알아보자



4.4 분산과 편차를 알아보자

- 작은 분산과 큰 분산의 의미

분산이 작다: 만약 분산값이 작다면, 이는 대부분의 학생들의 점수가 평균값 근처에 몰려 있다는 것을 의미한다. 즉, **학생들의 성적이 비교적 일관되고 비슷하다**는 뜻이다.

분산이 크다: 반면에 분산값이 크다면, 학생들의 점수가 평균값에서 넓게 퍼져 있다는 것을 나타낸다. 이는 학생들 사이의 성적 차이가 크다는 것을 의미할 수 있다.

- 이러한 분석을 통해 학급 내 학생들의 성적 분포를 더 잘 이해할 수 있음

- **분산**값은 단순히 **평균값만으로는 파악할 수 없는 데이터의 다양성과 변동성을 파악**하는 데 중요한 역할을 함

4.4 분산과 편차를 알아보자

- 분산을 구하는 방법은 각 관측값과 평균과의 차이를 계산하고, 이를 제곱한 다음 이들의 평균을 구하면 됨
- 여기서 각 관측값과 평균과의 차이를 **편차**라고 함

60점의 편차 제곱: $(60 - 80)^2 = 400$

70점의 편차 제곱: $(70 - 80)^2 = 100$

80점의 편차 제곱: $(80 - 80)^2 = 0$

90점의 편차 제곱: $(90 - 80)^2 = 100$

100점의 편차 제곱: $(100 - 80)^2 = 400$

$(400 + 100 + 0 + 100 + 400) / 5 = 1000 / 5 = 200$

- 따라서 A 모둠 점수 데이터의 분산은 200

4.4 분산과 편차를 알아보자

- 넘파이에서 제공하는 var() 함수는 지정된 축을 따라 분산을 구할 수 있음

```
print('A 모둠의 분산:', np.var(group_A))
```

```
A 모둠의 분산: 200.0
```

```
B = np.array([76, 78, 80, 82, 84])
print('B 모둠의 분산:', np.var(group_B))
```

```
B 모둠의 분산: 8.0
```

- A 모둠의 분산 값 200.0, B 모둠의 분산 값 8.0

- 이로부터 A 모둠 학생들의 점수 분포가 B 모둠에 비해 평균으로부터 떨어져 매우 넓게 분포하고 있다는 것을 알 수 있음

4.4 분산과 편차를 알아보자

- 분산을 통해서 데이터가 평균값 주변에 모여있는 정도를 측정할 수 있으나, 결국 이 값은 편차 제곱의 평균값이므로 **실제 데이터의 변동성을 나타내는 값과는 동떨어진 값**이 될 수 있음

- 따라서 이를 실제값과 근사시키기 위하여 분산에 제곱근을 씌워서 사용하며, 이 값을 **표준편차** standard deviation라고 함

- 표준편차를 구하는 넘파이 함수는 std() 함수를 사용

- m개의 인스턴스를 가진 데이터 x_i의 평균은 그리스 문자 μ_x로 표기하며, 분산은 σ_x^2으로, 표준편차는 σ_x 표기를 많이 사용

05 평균, 분산, 표준편차를 알아보자

- **평균과 편차, 분산, 표준편차는 데이터의 성질을 다루는 중요한 학문인 통계학에서 가장 기본적인 값들**이므로 잘 숙지해 두어야 함

용어	변량1	변량2	변량3	변량4	변량5	의미
변량	60	70	80	90	100	조사대상으로부터 얻은 같은 성질의 자료를 수량으로 나타낸 것
평균	$\frac{60+70+80+90+100}{5}=80$					주어진 변량의 합을 개수로 나눈 값으로 대표값 중의 하나임
편차	-20	-10	0	10	20	변량에서 평균을 뺀 값
편차제곱	400	100	0	100	400	편차값의 제곱값
분산	$\frac{400+100+0+100+400}{5}=\frac{1000}{5}=200$					편차의 제곱을 자료의 개수로 나눈값. 변량이 퍼진 정도
표준편차	$\sqrt{200}=14.1421$					분산값의 제곱근. 분산값을 실제값에 근사시키기 위하여 분산에 제곱근을 함

4.5 평균, 분산, 표준편차를 알아보자

- 앞서 살펴본 두 모둠 A, B의 표준편차를 구해보자.

```
print('A 모둠의 표준편차:', np.std(group_A))
```

```
A 모둠의 표준편차: 14.142135623730951
```

```
print('B 모둠의 표준편차:', np.std(group_B))
```

```
B 모둠의 표준편차: 2.8284271247461903
```

- A 모둠의 표준편차 값 14.14, B 모둠의 표준편차 값 2.83

- 이 결과를 통해서 A모둠 학생들은 개별 학생들의 성적 차이가 B 모둠에 비하여 크다는 것을 알 수 있음

4.5 평균, 분산, 표준편차를 알아보자

- A 모둠과 B 모둠의 표준편차를 분산의 제곱근을 이용하여 다시 계산하여 위의 결과와 동일함을 확인하자.
- 특정한 수나 다차원 배열의 제곱근은 넘파이 sqrt() 함수를 사용하여 구할 수 있음

```
print('A 모둠 분산의 제곱근 :', np.sqrt(np.var(group_A)))
```

```
A 모둠 분산의 제곱근 : 14.142135623730951
```

```
print('B 모둠 분산의 제곱근 :', np.sqrt(np.var(group_B)))
```

```
B 모둠 분산의 제곱근 : 2.8284271247461903
```

- 1에서 4까지의 정수를 원소로 하는 다차원 배열에 대하여 sqrt() 함수를 적용하여 제곱근으로 이루어진 다차원 배열을 구하는 코드도 작성해보자.

```
# 1, 2, 3, 4 를 원소로 하는 다차원 배열에 대한 제곱근 연산
np.sqrt(np.array([1, 2, 3, 4]))
```

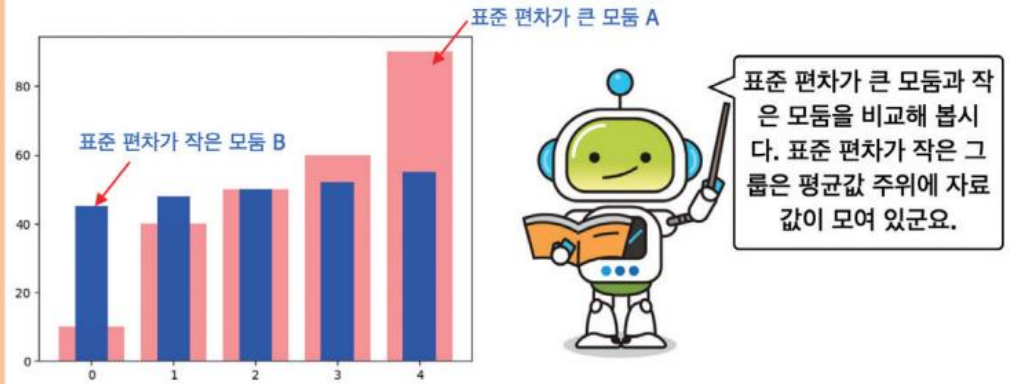
```
array([1.        , 1.41421356, 1.73205081, 2.        ])
```

4.5 평균, 분산, 표준편차를 알아보자

- A 모둠과 B 모둠 데이터의 분포를 시각화하는 방법으로 다음과 같이 matplotlib을 사용하도록 하자.

```
import matplotlib.pyplot as plt

x = np.arange(5)                    # x 구간의 5개 값 지정
plt.bar(x, group_A, color='red', alpha=.4)
plt.bar(x, group_B, color='blue', alpha=.9, width=.4)
```



- 표준편차가 큰 모둠 A는 폭이 넓은 분홍색, 표준편차가 작은 모둠 B는 폭이 좁은 파란색 막대
- 표준편차가 클수록 평균값 주위에서 멀리 떨어져 있는 자료들이 많고, 표준편차가 작을수록 평균값 근처에 자료들이 모여 있는 것을 확인할 수 있음

4.5 평균, 분산, 표준편차를 알아보자

도전문제 4.3 평균, 분산, 표준편차 구하기

난이도 중

다음과 같은 5개의 데이터에 대하여 편차, 평균, 분산, 표준편차 등을 구하여 표를 완성하여라.

용어	변량1	변량2	변량3	변량4	변량5
변량	76	78	80	83	83
평균	80				
편차	-4	-2	0	+3	+3
편차제곱	16	4	0	9	9
분산	7.6				
표준편차	2.75...				

LAB4-3 입력값의 평균, 분산, 표준편차 구하기

실습 시간

사용자로부터 임의의 정수를 입력받도록 하자. 이들 정수는 공백 단위로 구분된다. 이렇게 입력받은 정수값에 대하여 평균, 분산, 표준편차를 구하여 출력하여라.
이 기능은 넘파이의 함수를 사용하도록 하자.

원하는 결과

```
정수를 입력하시오 : 3 6 8 2 10 4 1
평균: 4.857142857142857
분산: 9.265306122448978
표준편차: 3.043896536094645
```

힌트

사용자로부터 공백 단위로 입력 받은 정수를 리스트에 저장하는 기능은 다음 코드를 사용하여 구현할 수 있다. 이 기능은 input() 함수로 입력받은 문자열을 split() 메소드로 구분한 다음 map 함수를 이용하여 정수형(int)으로 변환하는 기능이다.

```
arr = list(map(int,input('정수를 입력하시오 : ').split()))
```

```
정수를 입력하시오 : 3 6 8 2 10 4 1
```

이와 같은 명령으로 입력받은 정수값에 대하여 넘파이의 mean(), var(), std() 함수를 사용하면 이 기능을 구현할 수 있을 것이다.

LAB4-4 어느 모둠의 분산이 가장 큰가

실습 시간

다음과 같은 A, B, C, D 모둠에 대하여 수집한 키 데이터가 있다. 단위는 센티미터이다. 이 모둠들 중에서 분산이 가장 큰 모둠은 어느 모둠인가? 또한 분산이 가장 작은 모둠은 어느 모둠인가?

```
A = np.array([ 153, 186, 168, 170])
B = np.array([ 173, 175, 171, 176])
C = np.array([ 167, 189, 177, 188])
D = np.array([ 178, 154, 179, 174])
```

이 값을 다음과 같이 출력하여라.

원하는 결과

```
A 모둠의 분산 : 136.6875
B 모둠의 분산 : 3.6875
C 모둠의 분산 : 80.6875
D 모둠의 분산 : 102.6875
분산이 가장 작은 모둠은 B이며 분산은 3.6875
분산이 가장 큰 모둠은 A이며 분산은 136.6875
```

힌트

다음 코드는 모둠이름: 분산 값의 딕셔너리를 만드는 코드이다.

```
dic = {'A' : np.var(A),'B' : np.var(B),'C' : np.var(C),'D' : np.var(D)}
dic.items()
```

```
dict_items([('A', 136.6875), ('B', 3.6875), ('C', 80.6875), ('D', 102.6875)])
```

넘파이 명령은 아니지만 파이썬의 sorted() 함수는 튜플 형태의 입력값에 대하여 특정한 항목에 따라 정렬하는 기능이 있다. 예를 들어 다음 코드는 dic.items()의 튜플 값을 분산에 대하여 오름차순으로 정렬하여 new_dic이라는 새로운 튜플에 저장하는 기능이다.

```
new_dic = sorted(dic.items(), key=lambda x: x[1] )
```